

Mini-projet 07 — Héritage & classe abstraite : Plugin

Module B · POO · Notions : **héritage** · **super()** · **redéfinition** · **classe abstraite (ABC)** · **polymorphisme** · Durée : 2 h · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Ta boîte à outils aura plein de plugins : un qui scanne les ports, un qui lit les bannières, un qui ping... Ils sont tous **différents**, mais ils partagent une **structure commune** : un nom, et une action à lancer. Plutôt que réécrire cette structure dans chaque plugin, on la met une fois dans une classe **parente** `Plugin`, et chaque outil en **hérite**. C'est le **squelette** qui tiendra tout ton projet (et qu'on chargera dynamiquement au MP18).

1. L'héritage : réutiliser une classe

Une classe **enfant** hérite de tout ce que possède sa classe **parente** (attributs + méthodes), et peut en ajouter ou en modifier.

```
class Animal:
    def __init__(self, nom):
        self.nom = nom
    def decrire(self):
        return f"Je suis {self.nom}"

class Chien(Animal):          # Chien HÉRITE d'Animal
    def aboyer(self):
        return "Wouf"

c = Chien("Rex")
print(c.decrire())           # "Je suis Rex"  <- méthode HÉRITÉE d'Animal, gratuite
print(c.aboyer())            # "Wouf"         <- méthode propre à Chien
```

La syntaxe : `class Enfant(Parent):` . Tout ce qui est dans `Parent` est automatiquement disponible dans `Enfant` .

2. super() — appeler le constructeur du parent

Si l'enfant a besoin d'ajouter des attributs mais veut aussi ceux du parent, il appelle le constructeur parent avec `super()` :

```
class Chien(Animal):
    def __init__(self, nom, race):
        super().__init__(nom)    # "Parent, occupe-toi de self.nom"
        self.race = race         # puis j'ajoute ma propre donnée
```

`super()` = « la classe parente ». `super().__init__(nom)` exécute le `__init__` d' `Animal` , qui range `self.nom` . Sans ça, tu devrais recopier `self.nom = nom` à la main — l'héritage sert justement à éviter ça.

3. Redéfinir (override) une méthode

L'enfant peut **remplacer** une méthode du parent en la redéfinissant avec le même nom :

```
class Chat(Animal):
    def decrire(self):           # on ÉCRASE decrire d'Animal
        return f"Miaou, je suis {self.nom}"

Chat("Felix").decrire()         # "Miaou, je suis Felix"  (la version du Chat)
```

4. La classe abstraite — imposer un contrat notion clé

Problème : tous tes plugins **doivent** avoir une méthode `run()` . Comment **forcer** chaque plugin à l'implémenter, sinon erreur ?

Réponse : une **classe abstraite**. C'est un moule qu'on ne peut pas instancier directement, et qui exige que les enfants remplissent certaines méthodes.

```
from abc import ABC, abstractmethod
```

```
class Plugin(ABC):
    # hérite d'ABC = "je suis abstraite"
    def __init__(self, nom):
        self.nom = nom

    @abstractmethod
    def run(self, cible):
        # méthode abstraite : PAS de corps, juste le contrat
        ...

    def decrire(self):
        # méthode concrète : héritée telle quelle
        return f"Plugin: {self.nom}"
```

Ce que ça garantit :

```
Plugin("test")
# TypeError: Can't instantiate abstract class Plugin
# with abstract method run
```

Impossible de créer un `Plugin` « vide » : Python **refuse** tant qu'un enfant n'a pas fourni `run()` . C'est ça, le contrat.

Un plugin concret hérite de `Plugin` et **doit** définir `run()` :

```
class PluginPing(Plugin):
    def run(self, cible):
        return f"ping {cible.adresse()}"

p = PluginPing("ping")
print(p.decrire())
```

OK : run() est défini
"Plugin: ping" <- hérité de Plugin, gratuit

5. Le polymorphisme — la magie

Comme tous les plugins ont la même méthode `run()` , tu peux les traiter de façon **uniforme**, sans savoir lequel est lequel :

```
plugins = [PluginPing("ping"), PluginScan("scan")]
cible = Cible("10.0.0.5", 22)

for plugin in plugins:
    print(plugin.run(cible))
```

chaque plugin fait SA version de run()

Voilà tout l'intérêt : ta boîte à outils bouclera sur une liste de plugins et appellera `run()` sur chacun, **sans se soucier** de leur type. Ajouter un nouveau plugin = créer une classe qui hérite de `Plugin` . Le reste du code ne change pas. C'est extensible à l'infini.

Ta mission

Objectif : poser le socle `Plugin` abstrait et deux plugins concrets.

Prépare : recopie ta classe `Cible` du MP06 en haut du fichier (on verra les imports propres au MP18 — pour l'instant on la garde sous la main).

- Crée `class Plugin(ABC)` avec :
 - `__init__(self, nom)` qui range `self.nom` ;
 - une méthode **abstraite** `run(self, cible)` (avec `@abstractmethod` , sans corps) ;
 - une méthode **concrète** `decrire(self)` qui renvoie `"Plugin: <nom>"` .
- Crée `PluginPing(Plugin)` : son `run(self, cible)` renvoie `f"ping {cible.adresse()}"` .
- Crée `PluginScan(Plugin)` : son `run(self, cible)` renvoie `f"scan {cible.adresse()}"` .

Tests à faire passer :

```
cible = Cible("10.0.0.5", 22)

ping = PluginPing("ping")
print(ping.decrire())
print(ping.run(cible))

plugins = [PluginPing("ping"), PluginScan("scan")]
for p in plugins:
    print(p.run(cible))
```

-> Plugin: ping (méthode héritée)
-> ping 10.0.0.5:22
-> scan 10.0.0.5:22

```
# -> scan 10.0.0.5:22

# Et vérifie que ça, ça PLANTE bien (c'est voulu) :
# Plugin("vide") -> TypeError: Can't instantiate abstract class
```

Bonus : crée un 3e plugin `PluginBanniere` qui réutilise ton `decoder_banniere` du MP04 sur une bannière factice en bytes, et ajoute-le à la liste. Tu verras : la boucle marche sans la modifier.

- ✔ **Critères de réussite**
- `Plugin("x")` lève bien un `TypeError` (classe abstraite non instanciable).
 - `PluginPing` et `PluginScan` s'instancient et leur `run()` renvoie la bonne chaîne.
 - `decrire()` marche sur les deux **sans être réécrite** (elle est héritée).
 - La boucle `for p in plugins` appelle le bon `run()` sur chaque plugin.

Si tu bloques — indices progressifs

Étape	Indice
Import ABC	Tout en haut : <code>from abc import ABC, abstractmethod</code> .
Méthode abstraite	Le décorateur <code>@abstractmethod</code> juste au-dessus de <code>def run(self, cible):</code> , et dans le corps juste <code>...</code> (ou <code>pass</code>).
Plugin concret	<code>class PluginPing(Plugin):</code> — pas besoin de réécrire <code>__init__</code> , il est hérité. Tu définis juste <code>run</code> .
Appeler adresse()	Dans <code>run</code> , <code>cible</code> est un objet <code>Cible</code> → <code>cible.adresse()</code> te donne <code>"ip:port"</code> .

Où ranger : `mini-projets/07-plugin-abstrait/plugin.py` .
Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».